



15 ~ 16
set | 2026

Interconnectando

códigos dados inteligência inovação

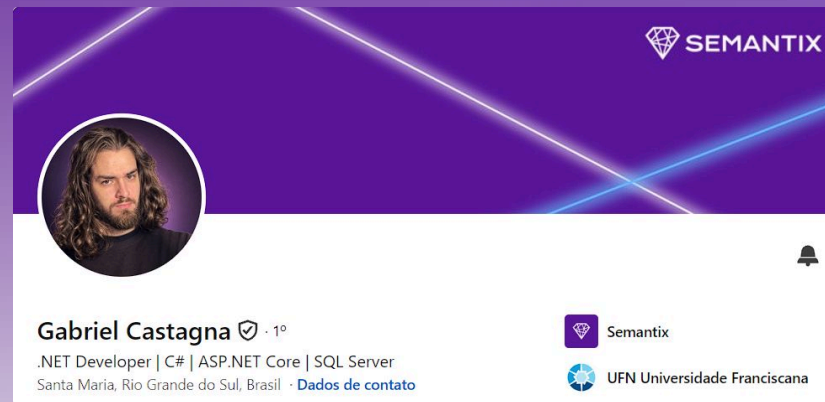


PyCaret

Automineração de Dados

Classificação de Churn com Python + PyCaret

Gabriel Castagna Henrique



<https://www.linkedin.com/in/gabriel-castagnah/>



<https://github.com/castagnagh/PyCaret>



O PROBLEMA

Podemos prever quais clientes têm maior probabilidade de cancelar?

Toda empresa de assinatura enfrenta o mesmo desafio: **clientes que cancelam sem aviso prévio**. Isso se chama **churn** — e o custo de perder um cliente é, em média, cinco vezes maior do que o custo de retê-lo.[1]

A pergunta de negócio é direta: *existe algum padrão nos dados que nos permita identificar, com antecedência, quem está prestes a cancelar?*

Se sim, podemos agir antes que a perda aconteça — com ofertas personalizadas, atendimento proativo ou ajustes de produto.

Custo real

Churn impacta receita, reputação e crescimento sustentável da empresa.

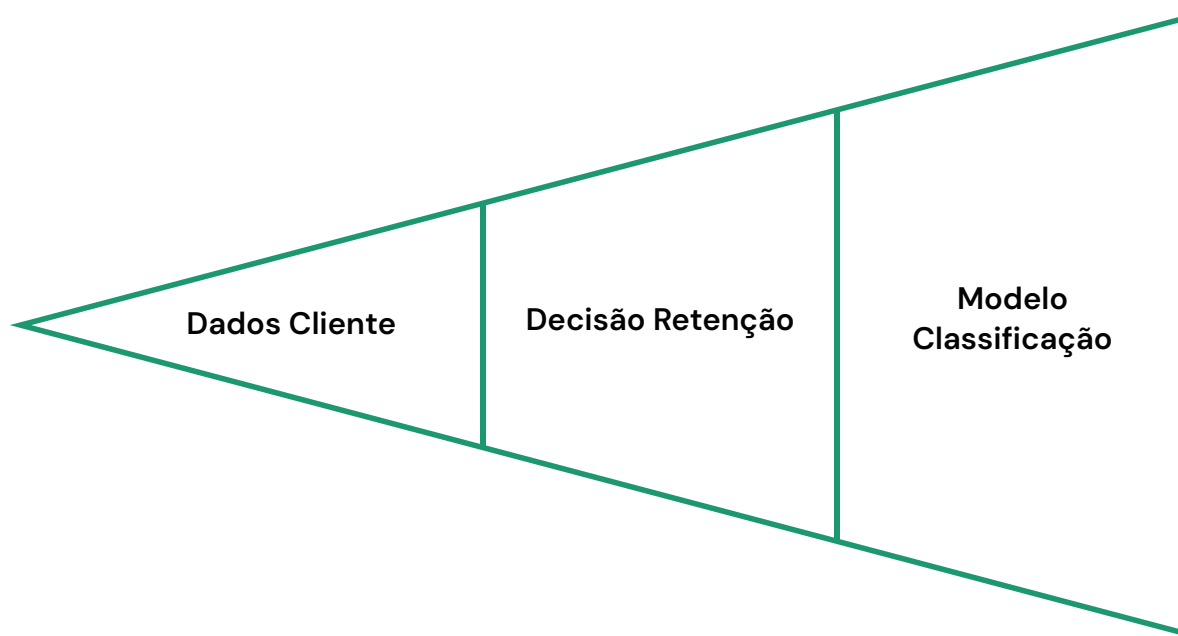
Oportunidade

Dados históricos de clientes contêm padrões que modelos de ML conseguem detectar.

O QUE VAMOS CONSTRUIR

Da informação bruta à decisão informada

Ao longo desta apresentação, construiremos um pipeline completo — do dado bruto ao modelo em produção.



Cada etapa tem um propósito claro. O modelo não toma a decisão final — ele **informa a decisão humana** com dados e probabilidades.

O que vamos aprender



O contexto

O problema de churn e por que ML é a abordagem certa



A ferramenta

O que é PyCaret, suas vantagens e suas limitações reais



Os dados

Dataset IBM Telco Churn — entendendo as features e o target



O experimento

Setup, comparação, tuning, avaliação e teste final



O deploy

Salvar o modelo e utilizá-lo em uma aplicação real

O que é PyCaret?

Definição

PyCaret é uma biblioteca de **Machine Learning de baixo código** para Python. Ela encapsula e orquestra ferramentas consolidadas como Scikit-learn, XGBoost, LightGBM, CatBoost e outras.

Com poucas linhas de código, você configura um ambiente de experimentos completo, compara múltiplos algoritmos e ajusta hiperparâmetros automaticamente.

PyCaret não é uma alternativa ao **Machine Learning tradicional**. É uma camada de automação construída sobre ele.

- Automatiza tarefas repetitivas de pré-processamento
- Compara múltiplos modelos com uma linha de código
- Reproduz experimentos com configuração centralizada
- Reduz tempo de experimentação, não de raciocínio

Por que Machine Learning pode ser trabalhoso?

Antes de entender o valor do PyCaret, é importante entender o problema que ele resolve. Um projeto de ML tradicional envolve **muitas etapas manuais e repetitivas**.

1

Coleta e limpeza

Tratar nulos, encoding, outliers — para cada dataset, do zero

2

Treinamento

Instanciar cada algoritmo manualmente, um por um

3

Comparação

Criar métricas, loops e tabelas comparativas manualmente

4

Tuning

Definir grids de hiperparâmetros e executar busca manual

Machine Learning Tradicional × PyCaret

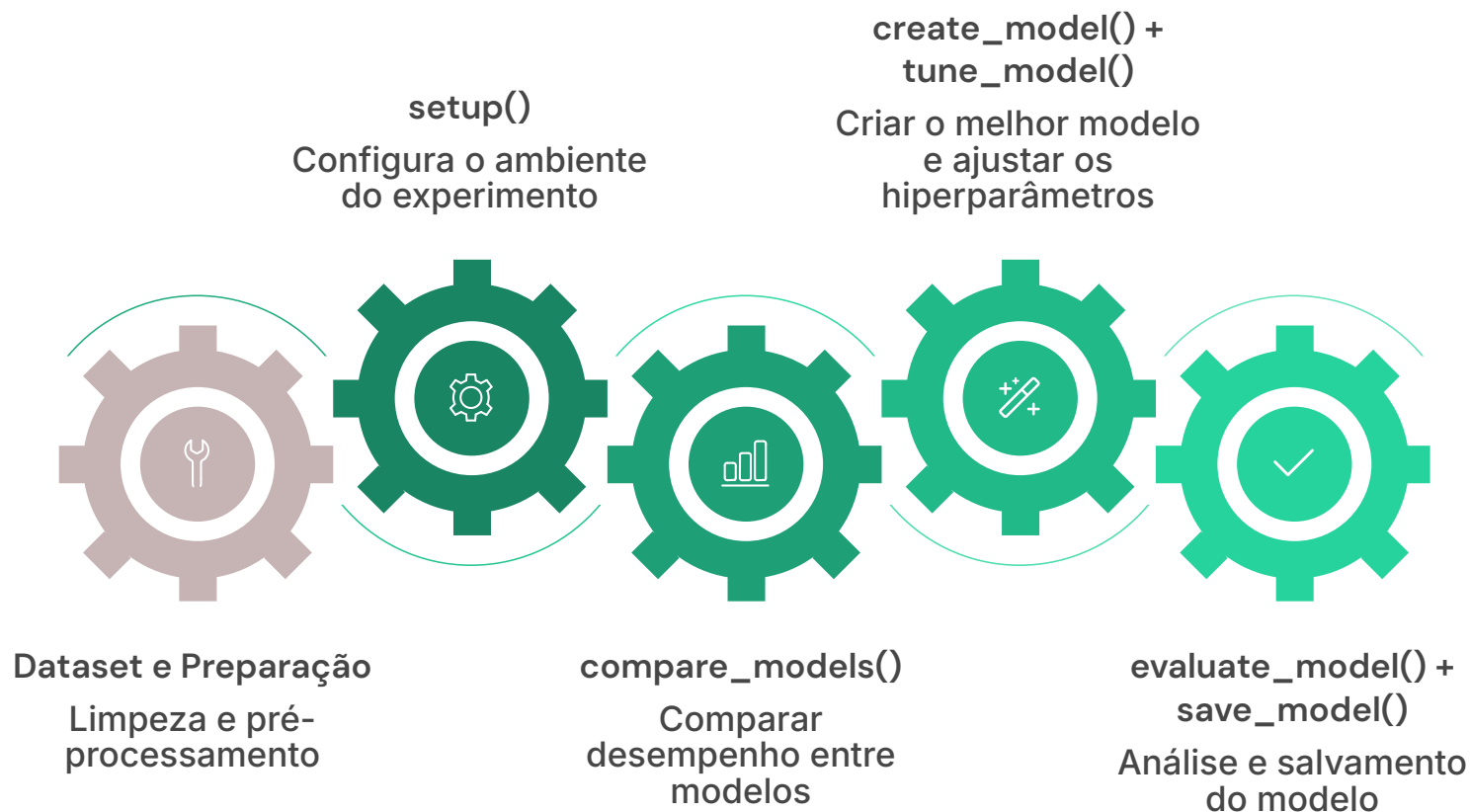
Etapa	ML Tradicional	Com PyCaret
Pré-processamento	Código manual extenso	<code>setup()</code> automatiza tudo
Comparar algoritmos	Loop com 10+ blocos de código	<code>compare_models()</code>
Tuning de hiperparâmetros	GridSearchCV manual	<code>tune_model()</code>
Avaliação visual	Plotar gráficos manualmente	<code>evaluate_model()</code>
Salvar modelo	joblib/pickle + metadados	<code>save_model()</code>



PyCaret não elimina a necessidade de entender o que cada etapa significa — ele elimina o trabalho mecânico de implementá-la do zero.

O fluxo completo do PyCaret

Cada função do PyCaret corresponde a uma etapa do ciclo de vida de um modelo de ML. Conhecer o fluxo é entender o projeto inteiro.



setup() — O ponto de partida de tudo

Código essencial

```
from pycaret.classification import *  
  
exp = setup(  
    data=df,  
    target='Churn',  
    session_id=42  
)
```

O que acontece internamente

- Divide os dados em treino e teste
- Trata valores ausentes automaticamente
- Aplica encoding em variáveis categóricas
- Normaliza ou escala features quando necessário
- Configura validação cruzada

Por que isso importa

O setup() é onde você **define as regras do experimento**. Tudo o que vier depois — comparação, tuning, avaliação — segue as configurações definidas aqui.

O parâmetro session_id garante **reprodutibilidade**: qualquer pessoa que rodar o mesmo código com o mesmo seed obterá os mesmos resultados.

❏ Erros no setup() se propagam para todo o experimento. Revise sempre as configurações antes de avançar.

compare_models() — Qual algoritmo se sai melhor?

Código essencial

```
best_model = compare_models(
    sort='AUC',
    n_select=3
)
```

O que acontece

PyCaret treina e avalia automaticamente **mais de 15 algoritmos** usando validação cruzada estratificada, retornando uma tabela comparativa com as principais métricas.

Por que isso importa

Em vez de escolher um único algoritmo "na intuição", você compara objetivamente múltiplos candidatos no mesmo conjunto de dados e sob as mesmas condições.

O parâmetro `sort='AUC'` ordena os modelos pelo AUC — métrica especialmente relevante para problemas de classificação com desbalanceamento de classes, como churn.

O resultado é uma **tabela ranqueada** dos melhores modelos, com métricas como Accuracy, AUC, Recall, Precision e F1.

create_model() + tune_model()

create_model()

```
rf = create_model(best_model)
```

Treina um modelo específico com os dados de treino e retorna os resultados de validação cruzada. Use para treinar o candidato selecionado na etapa anterior com controle total.

Aliases comuns

- 'rf' → Random Forest
- 'lightgbm' → LightGBM
- 'xgboost' → XGBoost
- 'lr' → Logistic Regression

tune_model()

```
tuned_rf = tune_model(  
    rf,  
    optimize='Recall'  
)
```

Executa busca automática de hiperparâmetros usando Random Search ou Optuna, otimizando a métrica especificada.

❏ Para churn, otimizar **Recall** é frequentemente a escolha certa — preferimos errar ao dizer que um cliente vai sair do que deixar passar alguém que vai cancelar.

evaluate_model() — Entendendo o comportamento do modelo

Código essencial

```
evaluate_model(tuned_rf)
```

Abre um painel interativo com múltiplas visualizações diagnósticas para análise profunda do modelo treinado.

Visualizações disponíveis

- Matriz de confusão
- Curva ROC e AUC
- Curva Precision-Recall
- Importância das features
- Curva de aprendizado

Por que não basta olhar o Accuracy?

Um modelo que prevê "não vai cancelar" para todos os clientes pode ter 85% de acurácia — se apenas 15% realmente cancelam. Mas esse modelo é **completamente inútil** para o negócio.

O `evaluate_model()` permite olhar além da acurácia e entender **onde o modelo erra**, com que frequência e para qual classe. Essa análise diagnóstica é o que separa um bom experimento de uma decisão bem fundamentada.

ATENÇÃO

PyCaret não faz tudo sozinho

Automatizar a experimentação não é o mesmo que automatizar o julgamento.

Você ainda precisa definir o problema

PyCaret não sabe se churn é mais custoso que um falso positivo. Você precisa traduzir o problema de negócio em escolhas técnicas.

Você ainda precisa analisar os dados

Nenhuma automação substitui a análise exploratória. Dados com problemas produzem modelos com problemas — independente da ferramenta.

Você ainda precisa interpretar as métricas

Qual métrica otimizar? Qual trade-off entre Precision e Recall é aceitável? Essas são decisões humanas com impacto real no negócio.

Você ainda precisa validar em produção

Um modelo bom no experimento pode falhar em dados reais. A validação em ambiente de produção exige atenção contínua.

Limitações do PyCaret

O que o PyCaret **não faz** por você

- **Análise exploratória de dados (EDA)** — necessária antes do setup
- **Engenharia de features criativa** — combinações de variáveis que fazem sentido de negócio
- **Tratamento de data leakage** — você precisa identificar e resolver antes
- **Interpretação do modelo para stakeholders** — isso é sua responsabilidade
- **Monitoramento em produção** — drift de dados não é detectado automaticamente

Riscos de uso sem critério

- Confiar cegamente no "melhor modelo" da tabela sem analisar suas características
- Ignorar o contexto de negócio ao escolher a métrica de otimização
- Usar PyCaret em datasets muito pequenos sem entender os limites da validação cruzada
- Tratar a automação como substituto do entendimento técnico



PyCaret facilita os experimentos. Não facilita pensar bem sobre o problema.

Data Leakage — o erro silencioso

O que é?

Data leakage ocorre quando informações que *não estariam disponíveis no momento da predição* são usadas durante o treinamento do modelo. O resultado: métricas de treino excelentes, desempenho real péssimo.

Exemplos comuns em churn

- Incluir a data de cancelamento como feature
- Usar variáveis calculadas após o evento de churn
- Vazar dados do conjunto de teste no pré-processamento

Por que o PyCaret não resolve isso?

O `setup()` divide os dados **depois** de recebê-los. Se o leakage já existia no DataFrame, ele será preservado no experimento.

A divisão treino/teste do PyCaret protege contra leakage *no pré-processamento* (transformações são fitadas apenas no treino), mas **não protege contra leakage nas features** — esse é sempre um problema de domínio e análise humana.

⊗ Um modelo com leakage parece extraordinário nos experimentos e falha completamente em produção.

Quando PyCaret faz sentido?

Ideal

- Dados tabulares estruturados
- Baseline rápido necessário
- Time pequeno
- Tempo limitado

Bom

- Experimento antes de ML customizado
- Ensino e didática
- Prototipagem

Cuidado

- Dados muito pequenos (abaixo de 500 linhas)
- Classes muito desbalanceadas

Não indicado

- Dados não tabulares (imagem, texto)
- Modelos customizados complexos
- Produção com requisitos de performance extremos

TRANSIÇÃO

Conhecemos o fluxo.

Agora precisamos aplicá-lo a um problema real.

Vamos trabalhar com o dataset **IBM Telco Customer Churn** — um conjunto de dados real de uma empresa de telecomunicações, com informações sobre contratos, serviços e comportamento de clientes que cancelaram ou permaneceram ativos.

Conhecendo o IBM Telco Customer Churn

Origem e escopo

Dataset público disponibilizado pela IBM, amplamente utilizado em competições e tutoriais de classificação binária. Representa clientes de uma empresa fictícia de telecomunicações.

7K+

Registros

Clientes únicos com histórico completo

21

Features

Variáveis descrevendo o perfil de cada cliente

~26%

Taxa de Churn

Proporção de clientes que cancelaram

Tipos de informação presentes

- **Demográficos:** gênero, parceiro, dependentes, idoso
- **Serviços:** telefone, internet, segurança online, suporte técnico, streaming
- **Contrato:** tipo (mensal, anual, bienal), faturamento eletrônico
- **Financeiros:** cobrança mensal, total acumulado
- **Engajamento:** tempo de relacionamento (tenure)

O target é a coluna Churn — binária: Yes ou No.

Features × Target

Features (variáveis preditoras)

São as informações que o modelo usa para aprender padrões. No nosso dataset:

- Tempo de contrato (tenure)
- Tipo de contrato (Contract)
- Valor mensal (MonthlyCharges)
- Tipo de internet (InternetService)
- Suporte técnico (TechSupport)

O modelo aprende como essas variáveis se relacionam com a decisão de cancelar.

Target (variável resposta)

É o que queremos prever. No nosso caso:

Churn = 'Yes' → Cliente **cancelou**

Churn = 'No' → Cliente permaneceu

Esta é uma classificação **binária**: o modelo atribui uma probabilidade para cada cliente pertencer à classe positiva (Churn = Yes).

❏ A classe positiva é, por convenção, o evento de interesse — neste caso, o cancelamento.

Carregando os dados

Código

```
import pandas as pd
from pycaret.datasets import get_data

# Carregando direto do repositório PyCaret
df = get_data('telco')

# Verificando estrutura básica
print(df.shape)      # (7043, 21)
print(df.dtypes)
print(df['Churn'].value_counts())
```

Alternativa — arquivo local

```
df = pd.read_csv('telco_churn.csv')
```

O que verificar imediatamente

- Dimensões do dataset (shape)
- Tipos de dados de cada coluna (dtypes)
- Distribuição do target — desbalanceamento?
- Valores ausentes (isnull().sum())
- Amostras do dado (head())

Este primeiro olhar é **insubstituível**. Nenhuma automação faz a EDA por você — e os problemas encontrados aqui afetam diretamente a qualidade do modelo.

Preparando os dados

Ajustes necessários antes do setup()

```
# TotalCharges foi importada como string
# por conter espaços em vez de NaN
df['TotalCharges'] = pd.to_numeric(
    df['TotalCharges'],
    errors='coerce'
)

# customerID não é uma feature útil
df = df.drop(columns=['customerID'])

# Verificar nulos após conversão
print(df.isnull().sum())
```

Por que fazer isso antes do PyCaret?

O PyCaret trata nulos automaticamente, mas ele não sabe que TotalCharges é numérica se estiver tipada como object. Tipos errados geram encoding incorreto.

Já a coluna customerID é um identificador único — se incluída, o modelo tentaria encontrar padrões nela, o que não faz sentido e pode causar overfitting.

 Preparação básica é sua responsabilidade. O setup() cuida do resto.

CONCEITO

Treino × Teste — por que separar?

Esta é uma das decisões mais importantes de qualquer projeto de ML. O PyCaret faz essa separação automaticamente no `setup()`, mas é fundamental entender *por que ela existe*.



⚠ Usar dados de teste durante o treinamento ou tuning invalida a avaliação. É como dar a resposta antes da prova.

Configurando o `setup()`

Código completo

```
from pycaret.classification import *  
  
exp = setup(  
    data=df,  
    target='Churn',  
    session_id=42,  
    train_size=0.7,  
    fix_imbalance=True,  
    normalize=True,  
    remove_outliers=False,  
    verbose=True  
)
```

Parâmetros importantes

`target='Churn'`

Define a variável resposta do experimento

`session_id=42`

Seed para reprodutibilidade total dos resultados

`fix_imbalance=True`

Aplica SMOTE para compensar o desbalanceamento de classes (~26% churn)

`normalize=True`

Normaliza features numéricas, beneficiando modelos como regressão logística e KNN

Comparando modelos com `compare_models()`

Código

```
top3 = compare_models(  
    sort='AUC',  
    n_select=3,  
    cross_validation=True,  
    fold=5  
)
```

O que acontece

PyCaret treina mais de 15 algoritmos com validação cruzada de 5 folds e retorna uma tabela ordenada por AUC. Os três melhores são armazenados em `top3`.

Tabela de resultados (exemplo de estrutura)

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
knn	K Neighbors Classifier	0.7720	0.7787	0.7720	0.7645	0.7669	0.3917	0.3939	0.2220
lr	Logistic Regression	0.7448	0.8352	0.7448	0.7438	0.6541	0.0727	0.1565	0.4360
ridge	Ridge Classifier	0.7416	0.7839	0.7416	0.7155	0.6534	0.0687	0.1358	0.0430

Qual modelo apresentou melhor desempenho?

Encontramos os candidatos. Agora vem uma decisão que o PyCaret não toma por você: qual modelo escolher e por quê.



Critério técnico — AUC

AUC responde "qual modelo consegue separar melhor quem cancela de quem não cancela?"



Critério de negócio — Recall

Recall responde "de todos os clientes que realmente iriam cancelar, quantos eu consegui identificar?"



Trade-off — Precision × Recall

Um Recall muito alto geralmente vem com Precision menor — mais clientes classificados como "churn" incluirão falsos positivos. Qual o custo de uma ação de retenção desnecessária?

- ❏ O modelo selecionado deve ser o melhor candidato considerando **tanto a métrica técnica quanto o contexto de negócio** — não apenas a primeira linha da tabela.

Treinamento e Tuning do modelo selecionado

Criando e ajustando

```
# Criar o modelo escolhido
# Substitua pelo alias do seu melhor modelo
modelo = create_model('[alias]')

# Tuning otimizando para Recall
tuned_modelo = tune_model(
    modelo,
    optimize='Recall',
    n_iter=50,
    search_library='optuna'
)
```

O que muda com o tuning?

O `tune_model()` explora o espaço de hiperparâmetros do modelo, tentando combinações que maximizam a métrica escolhida. Com `n_iter=50`, são testadas 50 configurações diferentes.

Como avaliar o modelo? Entendendo as métricas

Antes de executar o `evaluate_model()`, é importante entender o que cada métrica mede — especialmente no contexto de churn.

Accuracy

% de predições corretas no total.
Enganosa em datasets desbalanceados.

Precision

Dos que o modelo disse "vai cancelar", quantos realmente cancelaram? Evita alarmes falsos.

Recall ★

Dos que *realmente* cancelaram, quantos o modelo identificou?
Crítico para churn.

F1

Média harmônica entre Precision e Recall. Equilíbrio entre os dois.

AUC

Capacidade discriminativa geral do modelo. Independe do threshold escolhido.

📌 Para churn: priorize **Recall**. Falsos negativos — clientes que cancelam sem ser identificados — representam receita perdida sem chance de intervenção.

A Matriz de Confusão no contexto de Churn

Verdadeiro Negativo

VN — cliente não cancela, modelo diz não cancela.

CORRETO

Falso Positivo

FP — cliente não cancela, modelo diz cancela, ação de retenção desnecessária.

Falso Negativo

FN — cliente cancela, modelo diz não cancela.

PIOR ERRO — perda de receita

Verdadeiro Positivo

VP — cliente cancela, modelo diz cancela.

CORRETO — intervenção no momento certo

Falso Positivo (FP)

Oferecer desconto a quem não ia cancelar. Custo controlado — investimento desnecessário, mas a relação é preservada.

Falso Negativo (FN)

Não identificar um cliente que cancelará. Custo alto — perda de receita, sem chance de intervenção.

Por isso otimizamos **Recall**: queremos minimizar os FN, mesmo que isso aumente um pouco os FP.

Encontramos um modelo. Agora precisamos saber se ele realmente é bom.

Até agora, todas as métricas vieram da **validação cruzada no conjunto de treino**. O teste final usa dados que o modelo *nunca viu* durante o treinamento — essa é a avaliação que realmente importa.

Teste final — validando com dados não vistos

Finalizando e testando

```
# Finalizar: retreina com todos os dados
final_model = finalize_model(tuned_modelo)

# Avaliar no conjunto de teste reservado
predictions = predict_model(final_model)

# Inspecionar métricas finais
print(predictions[['Churn',
                  'prediction_label',
                  'prediction_score']].head(10))
```

O papel do `finalize_model()`

Antes do teste final, o `finalize_model()` retreina o modelo utilizando **todos os dados disponíveis** (treino + validação), maximizando a informação usada. Somente depois disso, o conjunto de teste é utilizado.

TRANSIÇÃO

Temos um modelo avaliado. Agora vamos colocá-lo em uma aplicação.

Um modelo que existe apenas em um notebook não gera valor. O próximo passo é salvar o modelo e integrá-lo a um fluxo que permita gerar previsões para novos clientes em tempo real ou em batch.

Salvando e utilizando o modelo

Salvando o modelo

```
# Salva o modelo e o pipeline completo
save_model(
    final_model,
    'modelo_churn_telco'
)
# Gera: modelo_churn_telco.pkl
```

Carregando em outro ambiente

```
modelo = load_model('modelo_churn_telco')

# Gerando previsões para novos clientes
novos_clientes = pd.read_csv('novos.csv')
resultado = predict_model(
    modelo,
    data=novos_clientes
)
```

O que é salvo junto com o modelo?

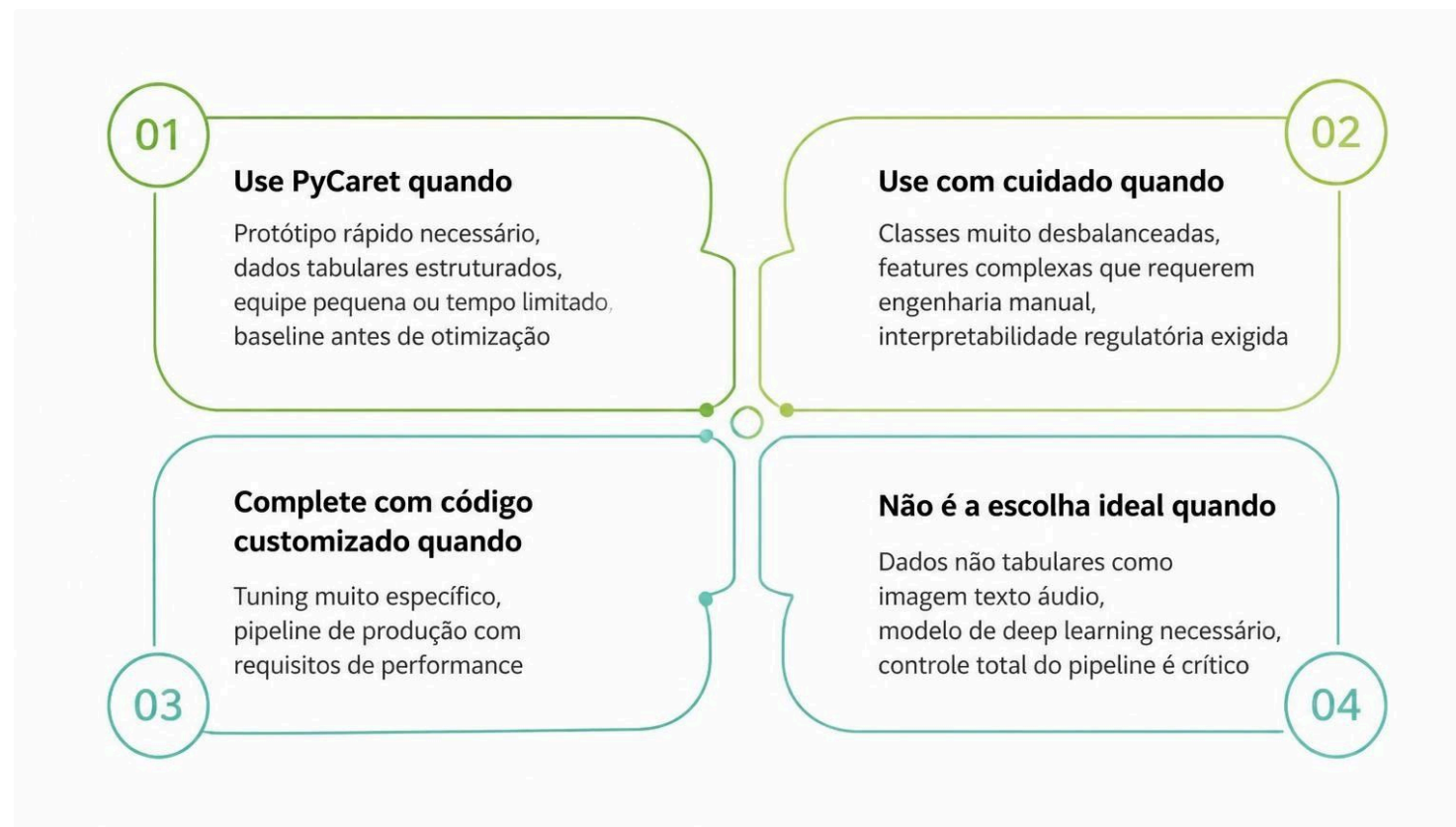
O `save_model()` persiste não apenas o algoritmo treinado, mas o **pipeline completo** de transformações definido no setup: encoding, normalização, tratamento de nulos — tudo é aplicado automaticamente aos novos dados.

- ✔ Isso elimina um dos problemas mais comuns em deploy: inconsistência entre o pré-processamento de treino e o de produção.

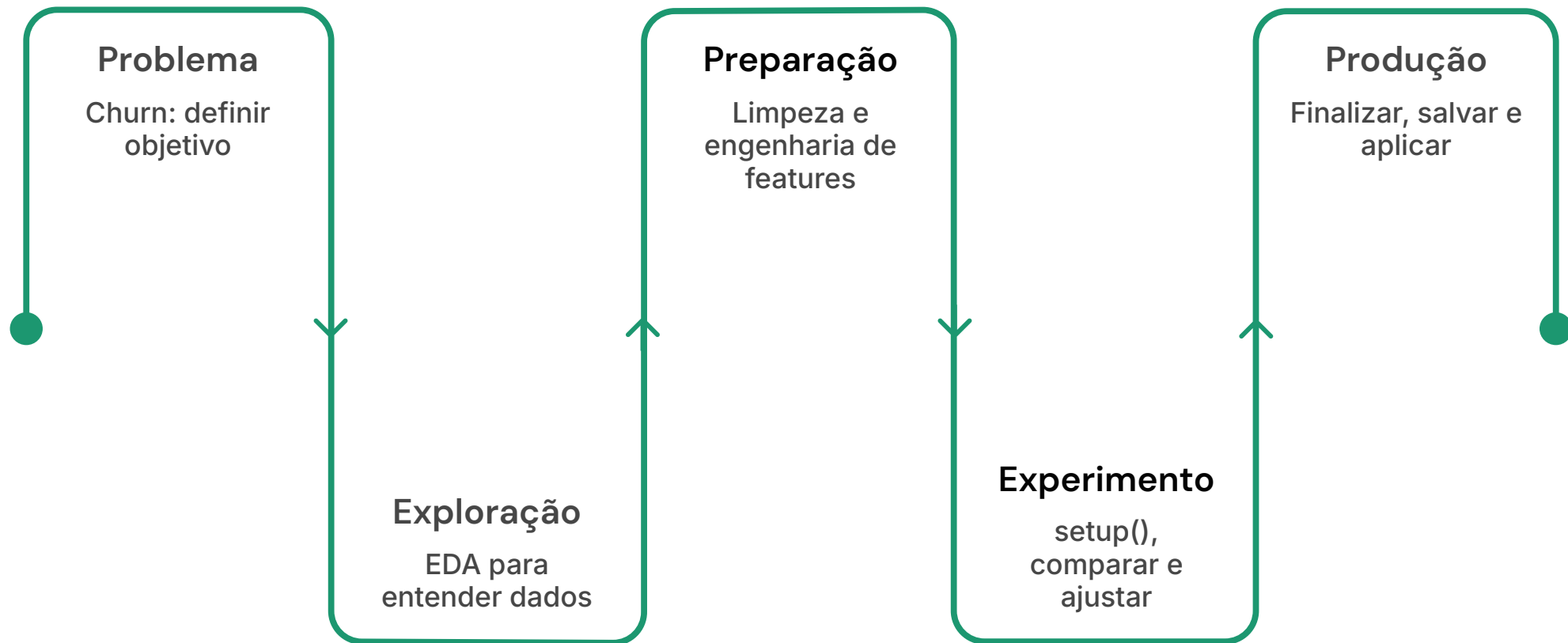
O que o resultado contém

- `prediction_label` — classe prevista (Yes/No)
- `prediction_score` — probabilidade de churn (0 a 1)

Quando usar PyCaret e quando considerar outra abordagem?



Do problema ao modelo em produção



O que você aprendeu hoje

1 Churn é um problema de classificação binária

Dados históricos de clientes contêm padrões que modelos de ML conseguem identificar — e PyCaret facilita enormemente a experimentação.

3 Métricas importam — escolha com critério

Para churn, Recall é a métrica mais crítica. Falsos negativos representam receita perdida sem possibilidade de intervenção.

2 O fluxo do PyCaret segue uma lógica clara

setup → compare → create → tune → evaluate → finalize → predict → save. Cada etapa existe por um motivo.

4 PyCaret automatiza a experimentação, não o julgamento

A ferramenta acelera o trabalho mecânico, mas todas as decisões importantes — métrica, modelo, threshold — requerem raciocínio humano e conhecimento de domínio.

CONCLUSÃO

A mensagem central

"PyCaret automatiza e simplifica a experimentação, mas não substitui o conhecimento necessário para tomar boas decisões de Machine Learning."

O que o PyCaret democratiza

- Acesso a múltiplos algoritmos sem código repetitivo
- Pré-processamento padronizado e reprodutível
- Experimentação rápida e comparação objetiva
- Pipeline de deploy consistente e completo

O que permanece sendo seu trabalho

- Entender o problema de negócio em profundidade
- Analisar os dados antes de qualquer automação
- Interpretar as métricas à luz do contexto real
- Garantir que o modelo seja justo, robusto e auditável

Perguntas?

Obrigado pela atenção.
